Secure Financial Platform: Study Guide

Implement BankTech's Compliance Framework — Module 2 hands-on practice

Overview

You are a Security Engineer at BankTech, a global bank launching an LLM-powered financial advisor for **5 million customers**. A penetration test exposed unsecured API endpoints, prompt-injection susceptibility and inadequate audit logging. You have **45 days** to implement a security framework that passes regulatory audits.

Learning objectives

#	Objective
LO1	Configure OAuth 2.0 authentication and IAM authorization for LLM API endpoints.
LO2	Implement rate limiting with an API gateway to prevent abuse and cost attacks.
LO3	Build prompt-injection defenses using input validation and sanitization.
LO4	Deploy output filtering to prevent PII leakage in model responses.
LO5	Establish audit logging that meets GDPR, SOC 2 and HIPAA requirements.

Tools and setup

- **Tools:** API gateway (Kong or AWS API Gateway), OWASP ZAP, ELK Stack (Elasticsearch, Logstash, Kibana), HashiCorp Vault, Python for middleware.
- **Setup:** Docker (to run Kong locally), Python 3.9+ with pip, OWASP ZAP, ELK via Docker Compose, at least 8 GB RAM.
- **Starter files** (Module2_Security): `M2_api_gateway_config.yaml` (Kong, rate-limit stubs), `M2_injection_tests.json` (18 test cases), `M2_prompt_validator.py`, `M2_audit_logger.py`, `M2_compliance_matrix.csv` (20 controls).

Activity 1 — API security controls (LO1, LO2)

Step	What to do
1. JWT authentication	Set up the JWT authorizer in the API gateway; validate issuer, audience and expiration; test with valid and invalid tokens.
2. IAM policies	Least-privilege role for the inference server (S3 read for model weights, CloudWatch write for logs); a separate role for the application calling the LLM endpoint.
3. Rate limiting	Per-user limits: 1,000 requests/hour (basic), 5,000 (professional), 20,000 (enterprise). Add response headers showing remaining quota.
4. Test	Unauthenticated requests → 401; limit exceeded → 429; document results as compliance evidence.

Tips: short token lifetimes; log all authentication failures; consider burst limits for legitimate traffic spikes.

Activity 2 — Prompt-injection defenses and output filtering (LO3, LO4)

Step	What to do
1. Input validation	Python middleware with regex patterns for common injection signatures; block phrases such as "ignore previous instructions" and similar.
2. Sanitization	Strip special characters and escape sequences; enforce length limits (course target: 4,096 tokens); protect the system prompt against override.
3. PII output filter	Regex detection for SSN, credit cards and similar; redact account numbers and balances; block the response when leakage is detected.
4. Test	Run the provided injection tests; confirm every attack is blocked and logged; confirm PII is caught in responses; document results.

Tips: layer defenses — no single control is sufficient; log blocked requests without storing the malicious content; consider NeMo Guardrails for production.

Activity 3 — Compliance-ready audit logging (LO5)

Step	What to do
1. Log schema	Structured format with timestamp, user_id, session_id; prompt **hash** (not full content), response status, token count; source IP, user agent, request duration.
2. ELK stack	Deploy with Docker Compose; Logstash pipeline to parse and index LLM logs; retention policies (course: 7 years for HIPAA, 6 years for SOC 2).
3. Compliance controls	GDPR right-to-deletion via pseudonymization; role-based access to logs; encryption at rest and in transit.
4. Documentation	Map logging controls to SOC 2, GDPR and HIPAA; sample audit queries for auditors; retention and deletion procedures.

Tips: never log full prompts — hashes protect IP; separate security events from operational metrics; alert on anomalous access patterns.

The compliance matrix at a glance

The starter matrix lists **20** controls: SOC 2 10, GDPR 4, PCI-DSS 3, OWASP LLM 3. Risk levels: Critical 5, High 11, Medium 4. At the start, 19 are TODO and 1 is already implemented (data minimization via prompt hashing).

What to submit (peer review)

Requirement	Include
1. API authentication	Gateway config with JWT; IAM least-privilege policies; evidence of 401 rejections; token validation results.
2. Rate limiting	Three-tier configuration; evidence of 429 responses; quota headers; burst-limit configuration.
3. Prompt-injection defense	Input validation middleware; sanitization with regex; test results of blocked attempts; security-event logging config.
4. PII output filtering	PII pattern configuration; financial redaction rules; detection/blocking test results; redaction examples.
5. Audit logging	Log schema and ELK config; compliance matrix (SOC 2, GDPR, HIPAA); sample audit queries; 3–5 sentence business analysis.

Key takeaways

- LLM security needs layers: authentication, input validation, output filtering and audit logging work together.
- Prompt injection is specific to LLM applications and needs defenses beyond traditional API security (OWASP Top 10 for LLMs).
- Compliance-ready audit logging, with retention policies and access controls, is essential in regulated industries — the course cites 1,100+ exposed Ollama servers found by Cisco Talos.

Quick self-check

- Which JWT claims does the gateway verify, and what is the maximum token lifetime in the reference config?
- Why store a prompt hash instead of the prompt?
- What HTTP status do over-limit requests receive?
- Why is regex-only input validation not enough on its own?